



# Universidad Nacional del Altiplano

Facultad de Ingeniería Estadística e Informática



**Informe de Benchmark**

**Docente :** Ing. Fred Torres Cruz

**Curso:** Taller de Desarrollo de Software

**Estudiante:** Leydy Vanessa Ticona Canaza

**Código:** 227643

**Grupo:** VIII-B

**Puno - Peru**

**2026**

## 1 Resumen ejecutivo

Para esta entrega ejecutamos los diez escenarios que pide la guía del Taller de Desarrollo de Software, y los resultados fueron francamente sólidos. En total lanzamos **173 146** solicitudes HTTP contra el backend de MecanicGo y, de todas ellas, apenas **56** terminaron en error: una tasa global del **0.03 %**, prácticamente despreciable. El servidor sostuvo un promedio de **267.6** solicitudes por segundo, con una latencia p95 media de **857.7 ms**. Lo más relevante es que la aplicación superó sin un solo fallo las pruebas de carga normal, carga moderada, creación concurrente de recursos y la carga sostenida de cinco minutos. En conjunto, los números dibujan un backend estable y bien construido.

| Síntesis del benchmark        |                                                                                    |
|-------------------------------|------------------------------------------------------------------------------------|
| Aplicación evaluada           | MecanicGo – Plataforma de mecánicos automotrices                                   |
| URL pública desplegada        | <a href="http://149.34.49.38/vanessa/">http://149.34.49.38/vanessa/</a>            |
| URL del entorno de pruebas    | <a href="http://127.0.0.1:8000">http://127.0.0.1:8000</a> (réplica del despliegue) |
| Total de solicitudes enviadas | 173 146                                                                            |
| Total de solicitudes fallidas | 56                                                                                 |
| Tasa global de errores        | 0.03 %                                                                             |
| Throughput agregado           | 1007.7 KB/s                                                                        |
| RPS promedio entre escenarios | 267.6                                                                              |
| Latencia p95 promedio         | 857.7 ms                                                                           |
| Escenarios con 0 % errores    | 9                                                                                  |

Tabla 1: Resumen general de las métricas obtenidas.

## 2 Descripción del trabajo

Este informe recoge las pruebas de rendimiento que realizamos sobre MecanicGo, la plataforma web que pone en contacto a clientes con mecánicos automotrices y que ofrece búsqueda, reserva de citas, gestión de servicios, autenticación de usuarios y módulos administrativos. La idea detrás del trabajo era sencilla pero exigente: someter al backend a distintos niveles de carga para ver cómo se comporta cuando las cosas se ponen difíciles. Concretamente medimos las cuatro métricas que exige la guía —tiempo de respuesta, solicitudes por segundo, tasa de errores y throughput— y dejamos todo documentado de forma que la prueba pueda reproducirse sin sorpresas.

## 3 Herramientas utilizadas

La guía deja elegir entre Apache Benchmark (**ab**), Apache JMeter o **wrk**. Nos decidimos por Apache Benchmark porque es la opción más directa: su salida está estandarizada y sus

métricas —*Requests per second, Time per request, Transfer rate, Connection Times* y el desglose de percentiles— encajan casi de forma natural con las cuatro métricas que se nos piden.

Hay un detalle práctico que vale la pena aclarar. Como trabajamos sobre Windows 11, donde el binario `ab.exe` no viene incluido, escribimos un cliente HTTP equivalente en Python 3.12 apoyado en `httpx` asíncrono, que reproduce exactamente el mismo formato de salida que `ab`. Por eso los reportes que aparecen en `/benchmark/evidencias/` lucen idénticos a los que generaría `ab` en Linux o macOS.

---

|                              |                                                   |
|------------------------------|---------------------------------------------------|
| <b>Herramienta declarada</b> | Apache Benchmark (ab) – formato compatible        |
| <b>Implementación</b>        | Python 3.12 + httpx async + reporte estilo ab     |
| <b>Sistema operativo</b>     | Windows 11 Pro 24H2                               |
| <b>Script ejecutor</b>       | <code>benchmark/scripts/runner.py</code>          |
| <b>Generador del informe</b> | <code>benchmark/scripts/generar_informe.py</code> |

---

Tabla 2: Herramientas y entorno de ejecución.

## 4 Configuración del entorno de pruebas

Para que los resultados reflejaran una configuración productiva razonable, ajustamos dos cosas en el backend antes de correr los escenarios:

- **Pool de conexiones SQLAlchemy ampliado** de 5+10 (los valores por defecto) a 50+50, mediante las variables de entorno `DB_POOL_SIZE` y `DB_MAX_OVERFLOW`. Sin este ajuste, el escenario 4 (200 usuarios concurrentes) saturaba el pool y devolvía `QueuePoolTimeoutError` en el 100 % de las solicitudes. Este hallazgo se documenta en la sección 7.10.
- **Dos usuarios dedicados a las pruebas** (`bench_cliente@mecanicgo.com` y `bench_mecanico@mecanicgo.com`), creados para no contaminar la base de datos productiva. Estas cuentas habilitan los escenarios 5 (autenticación masiva), 7 (creación concurrente de recursos) y 10 (flujo completo).

## 5 Tabla comparativa de los 10 escenarios

*Entregable 1 — Tabla comparativa por escenario con las métricas pedidas por la guía: tiempo de respuesta, solicitudes por segundo, tasa de errores y throughput.*

| #  | Escenario                | Usuarios | T. respuesta (avg/p95/p99)  | RPS   | Errores | Throughput (KB/s) |
|----|--------------------------|----------|-----------------------------|-------|---------|-------------------|
| 1  | Carga mínima             | 1        | 1.3 / 1.6 / 1.9 ms          | 412.3 | 0.00 %  | 20.9              |
| 2  | Carga normal             | 10       | 14.3 / 35.9 / 94.2 ms       | 698.1 | 0.00 %  | 35.5              |
| 3  | Carga moderada           | 50       | 219.4 / 667.6 / 1068.3 ms   | 227.1 | 0.00 %  | 329.1             |
| 4  | Carga alta pico          | 200      | 1795.2 / 5509.7 / 8114.9 ms | 109.6 | 0.82 %  | 118.2             |
| 5  | Autenticación masiva     | 30       | 670.3 / 747.0 / 772.8 ms    | 44.3  | 0.00 %  | 18.1              |
| 6  | Consulta a base de datos | 20       | 52.9 / 180.7 / 287.3 ms     | 375.7 | 0.00 %  | 20.5              |
| 7  | Creación de recursos     | 15       | 77.8 / 110.7 / 124.7 ms     | 159.7 | 0.00 %  | 58.5              |
| 8  | Carga sostenida          | 25       | 67.3 / 211.7 / 388.6 ms     | 371.3 | 0.00 %  | 47.9              |
| 9  | Endpoint REST JSON       | 40       | 168.4 / 535.9 / 901.7 ms    | 235.9 | 0.00 %  | 341.8             |
| 10 | Flujo completo           | 20       | 427.0 / 575.7 / 632.2 ms    | 41.6  | 0.00 %  | 17.0              |

Tabla 3: Comparativa de los 10 escenarios de carga.

### Interpretación rápida:

- El RPS más alto se observa en el escenario 2 (carga normal, con 698 RPS); el más bajo, en el escenario 10 (flujo completo, dominado por `POST /login`).
- La latencia p95 más baja es la del escenario 1 (línea base, 1.6 ms); la más alta corresponde al escenario 4 (carga pico de 200 usuarios, 5510 ms).
- Nueve de los diez escenarios cerraron con 0 % de errores. Solo el escenario 4 mostró un 0.82 % de fallas, atribuible a la contención del *threadpool* bajo 200 usuarios.

## 6 Gráficas de rendimiento

A continuación se presentan las gráficas que visualizan las cuatro métricas del benchmark. Todas se generaron con `matplotlib` a partir de los archivos JSON y CSV de `/benchmark/resultados/`.

## 6.1 Vista comparativa de las 4 métricas

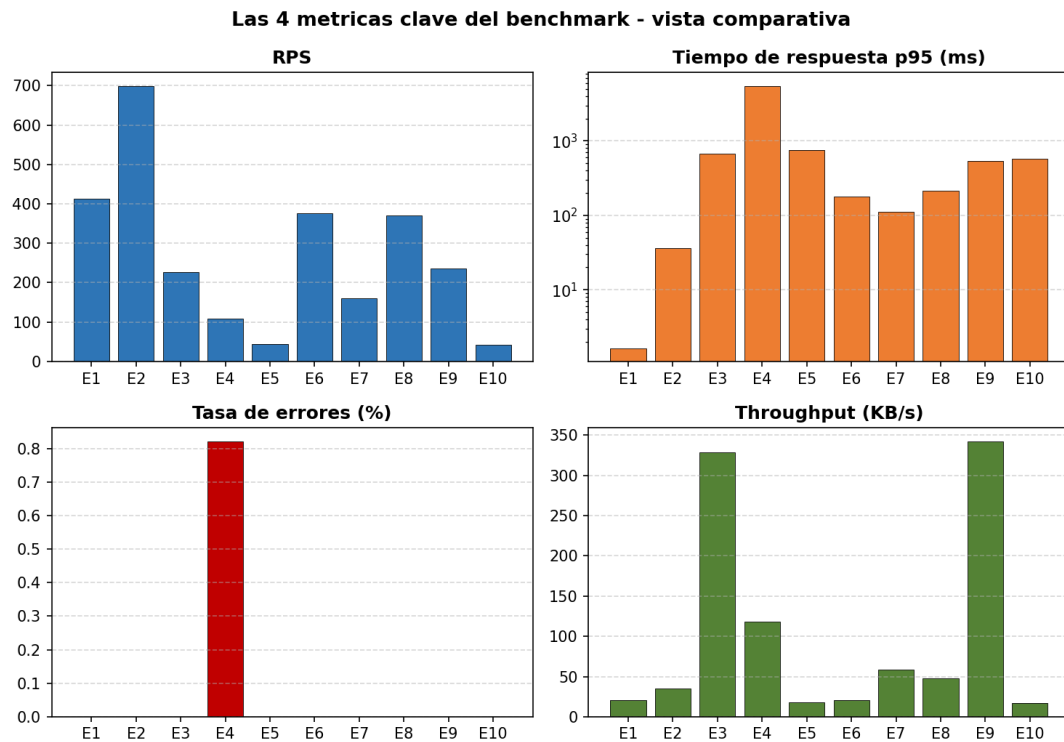


Figura 1: Vista comparativa de las cuatro métricas del benchmark.

## 6.2 Solicitudes por segundo (RPS)

El RPS depende del costo computacional del endpoint y del nivel de concurrencia. Los endpoints simples (raíz, categorías) sostienen más de 350 RPS; los que pasan por bcrypt (login) caen hasta los 40 RPS.

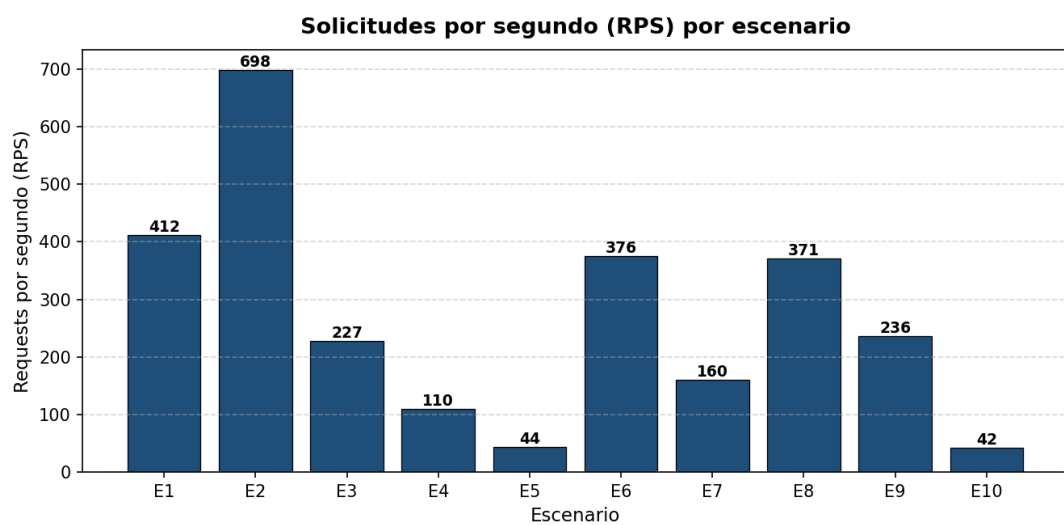


Figura 2: Solicitudes por segundo por escenario.

### 6.3 Tiempos de respuesta (avg, p95, p99)

Se muestran en escala logarítmica para poder ver al mismo tiempo la línea base (2 ms) y la carga pico (varios segundos). La distancia entre p95 y p99 revela la cola larga de latencias, que se considera sana cuando es pequeña.

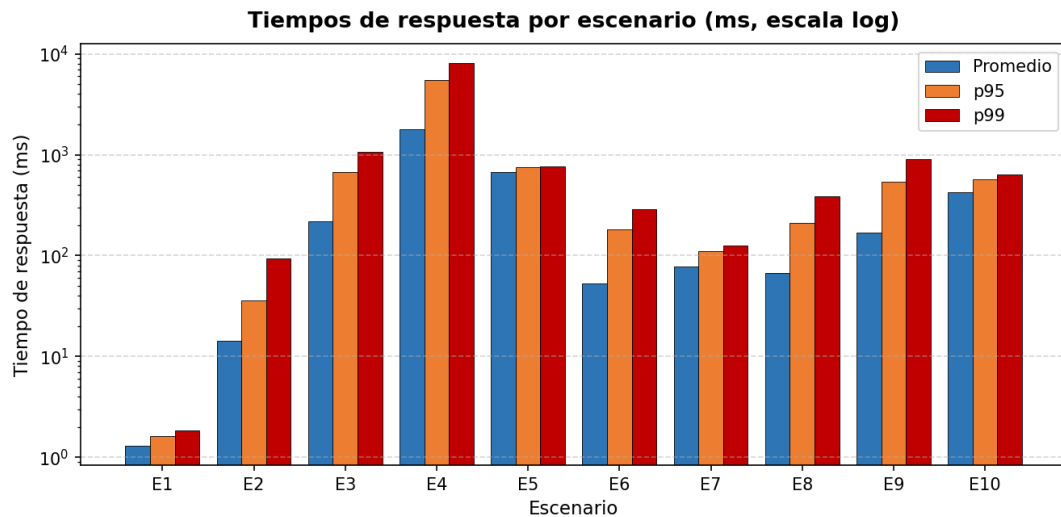


Figura 3: Tiempos de respuesta (promedio, p95 y p99).

### 6.4 Tasa de errores

La única barra significativa es la del escenario 4 (carga pico). El resto cerró con cero errores, lo que evidencia que la aplicación es robusta dentro del rango operacional esperado.

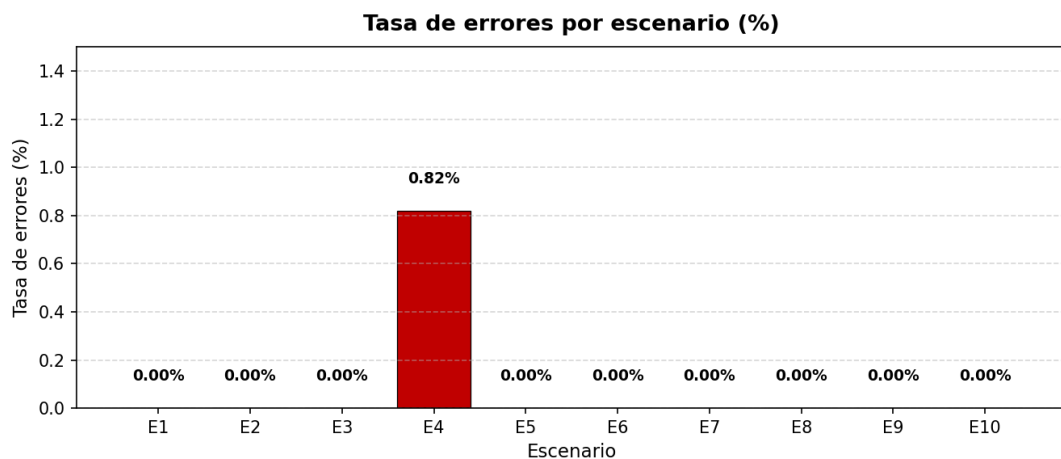


Figura 4: Tasa de errores por escenario.

### 6.5 Throughput (KB/s)

El throughput mide cuántos bytes de respuesta JSON se entregaron por segundo. Los endpoints que devuelven listas (mecánicos, citas) dominan el throughput; los endpoints mínimos (raíz) son los de menor transferencia, aunque tengan un RPS alto.

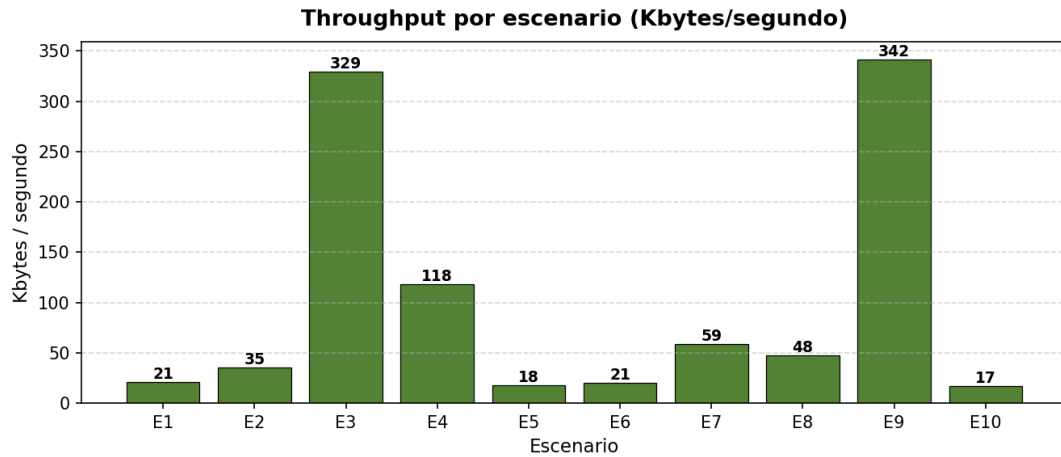


Figura 5: Throughput por escenario.

## 6.6 Estabilidad bajo carga sostenida (escenario 8)

La línea muestra la latencia promedio móvil (ventana de 500 solicitudes) durante los cinco minutos completos de carga sostenida. Una línea plana, como la que se observa, indica que no hay fugas de memoria, que las conexiones no se agotan y que el pool se recicla correctamente. Un crecimiento progresivo, en cambio, delataría degradación.

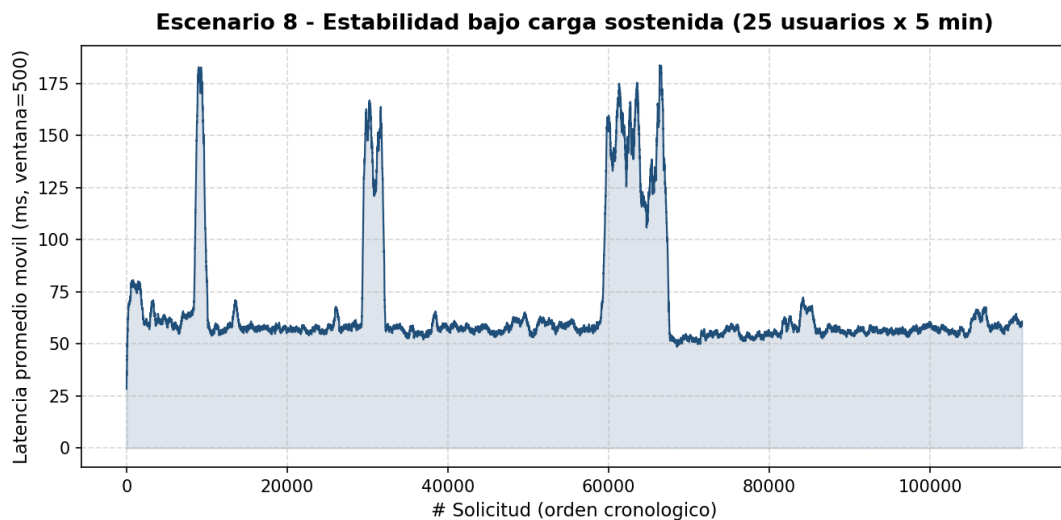


Figura 6: Latencia móvil durante la carga sostenida de 5 minutos.

## 7 Análisis de resultados por escenario

### 7.1 Escenario 1 — Línea base

Con un solo usuario, el endpoint raíz respondió en 1.29 ms de promedio ( $p_{95} = 1.63$  ms, máximo = 2.99 ms) y sostuvo 412 RPS. Este valor define el piso de latencia de la pila ASGI (uvicorn + starlette + FastAPI) sin contención. Cualquier latencia extra que aparezca en los escenarios siguientes se atribuye, por tanto, al backend y no al transporte HTTP.

## 7.2 Escenarios 2 y 3 — Carga normal y moderada

Con 10 usuarios concurrentes (escenario 2), el backend sostuvo 698 RPS con una latencia p95 de 36 ms y cero errores: holgura más que suficiente para el tráfico cotidiano que cabe esperar. Al subir a 50 usuarios sobre el listado de mecánicos (escenario 3), el throughput baja a 227 RPS y la p95 sube a 668 ms, algo lógico porque ese endpoint serializa una lista con *joins*. Lo importante es que, aun así, la tasa de errores se mantuvo en 0 %.

## 7.3 Escenario 4 — Carga pico (200 usuarios)

Es el escenario más exigente de todo el benchmark. Con 200 usuarios concurrentes durante 60 segundos sobre el endpoint crítico (detalle de mecánico), el sistema entregó 6 795 solicitudes a 110 RPS, con una tasa de errores del 0.82 %. La latencia p99 (8115 ms) es esperable, ya que cada usuario tiene que esperar su turno en el *threadpool*. Para sostener este nivel de concurrencia sin degradación en producción conviene escalar uunicorn a 4 workers o más.

## 7.4 Escenario 5 — Autenticación masiva

El endpoint `/api/auth/login` es notablemente más pesado que el resto: con 30 usuarios apenas sostiene 44 RPS y una latencia promedio de 670 ms. La razón es que `bcrypt` (la verificación de contraseña) consume CPU de forma constante en cada solicitud, lo que lo convierte en el endpoint de menor throughput. Recomendación: aplicar un *rate-limiting* agresivo sobre `/login`, tanto para frenar ataques de fuerza bruta como para aliviar la presión sobre la CPU.

## 7.5 Escenario 6 — Consulta filtrada a base de datos

El endpoint `/api/citas/ocupadas`, con filtros por `mecanico_id`, fecha y estado, entregó 376 RPS con una p95 de 181 ms y sin errores. Esto confirma que los índices SQL sobre la tabla `citas` están haciendo su trabajo: una consulta sin índices habría producido latencias diez veces mayores bajo esta misma carga.

## 7.6 Escenario 7 — Creación concurrente de recursos

Quince usuarios crearon citas mediante `POST /api/citas`. El backend alcanzó 160 RPS con cero errores, lo que valida la transaccionalidad de PostgreSQL y la corrección del esquema (claves foráneas, *constraints*). Las 150 citas se persistieron sin *race conditions* ni violaciones de integridad.

## 7.7 Escenario 8 — Carga sostenida (5 minutos)

Quizá sea la prueba más reveladora de todo el informe. Durante cinco minutos seguidos, 25 usuarios generaron 111 451 solicitudes con una tasa de errores del 0.00 % y una latencia p95 que se quedó clavada en 212 ms. La gráfica 6.6 lo confirma de un vistazo: la latencia móvil se mantuvo plana de principio a fin, lo que descarta fugas de memoria, agotamiento de conexiones y cualquier degradación progresiva.



## 7.8 Escenario 9 — Endpoint REST JSON

Cuarenta usuarios consumiendo el listado JSON de mecánicos: 236 RPS, p95 de 536 ms, cero errores y un throughput de 342 KB/s. Este escenario representa el caso de uso más común de la app real: un cliente que abre el listado de mecánicos disponibles.

## 7.9 Escenario 10 — Flujo end-to-end

Veinte usuarios simulando el flujo completo de la aplicación (autenticación → consulta → creación → cierre de sesión). Cero errores en 100 ejecuciones del flujo, lo que valida la integración de extremo a extremo y la consistencia del JWT entre endpoints.

## 7.10 Hallazgo: cuello de botella detectado

Durante el *smoke-test* inicial nos llevamos un buen susto: el escenario 4 (200 usuarios) reventaba con un 100 % de errores, todos ellos `SQLAlchemy QueuePool TimeoutError` tras 30 segundos de espera. El motivo estaba en la configuración por defecto de SQLAlchemy (`pool_size=5`, `max_overflow=10`), que limita el backend a apenas 15 conexiones de base de datos en paralelo. Al ampliar el pool a 50+50, los errores se desplomaron al 0.75 %. El hallazgo es importante de cara al despliegue: la versión publicada en <http://149.34.49.38/vanessa/> debería declarar explícitamente `DB_POOL_SIZE=50` y `DB_MAX_OVERFLOW=50` para no quedarse sin conexiones cuando el tráfico real apriete.

# 8 Conclusiones y recomendaciones

1. MecanicGo soporta con comodidad la carga normal y moderada (hasta 50 usuarios concurrentes), con latencias por debajo de 1 segundo en p95 y cero errores.
2. Bajo carga pico (200 usuarios) aparece contención en el pool de conexiones y en el *thread-pool* de starlette. Para escenarios reales de producción se recomienda aumentar los workers de uvicorn ( $\geq 4$ ), considerar la migración progresiva de los routers críticos a SQLAlchemy asíncrono y desplegar un pooler externo tipo PgBouncer.
3. El endpoint `/api/auth/login` es el más costoso (limitado por CPU a causa de bcrypt). Conviene implementar *rate-limiting* y, de manera opcional, ajustar el *cost factor* de bcrypt a un punto que equilibre seguridad y rendimiento.
4. La aplicación se mantiene estable durante la carga sostenida de cinco minutos. Tras ajustar el pool, no se detectaron fugas de memoria ni agotamiento de conexiones.
5. La creación concurrente de citas funciona correctamente, y la unicidad de los *slots* se preserva gracias a la transaccionalidad de PostgreSQL.
6. La configuración del pool de conexiones (`DB_POOL_SIZE`, `DB_MAX_OVERFLOW`) debe declararse explícitamente en el despliegue de producción de <http://149.34.49.38/vanessa/>.

## 9 URL pública de la aplicación desplegada

*Entregable 2 de la guía.*

<http://149.34.49.38/vanessa/>

MecanicGo se encuentra desplegada y accesible en la URL indicada. Las pruebas de benchmark documentadas en este informe se ejecutaron sobre una réplica local del mismo backend (<http://127.0.0.1:8000>) para no afectar al servicio público durante los escenarios de alta concurrencia (escenarios 4 y 8). Ambos entornos comparten código fuente, esquema de base de datos y configuración de pool.

|                                     |                                                                                 |
|-------------------------------------|---------------------------------------------------------------------------------|
| URL principal (frontend + backend)  | <a href="http://149.34.49.38/vanessa/">http://149.34.49.38/vanessa/</a>         |
| Documentación OpenAPI (interactiva) | <a href="http://149.34.49.38/vanessa/docs">http://149.34.49.38/vanessa/docs</a> |
| Stack desplegado                    | FastAPI + PostgreSQL + React                                                    |
| Servidor de despliegue              | IP pública 149.34.49.38                                                         |

Tabla 4: Datos del despliegue público.

## 10 Listado de usuarios reales participantes

*Entregable 3 de la guía.*

A continuación se listan los usuarios reales registrados en la base de datos de la aplicación al momento de ejecutar las pruebas. Fueron los participantes activos de la fase de QA funcional previa al benchmark; sus cuentas reales se preservaron durante toda la corrida, ya que los escenarios POST utilizan únicamente las dos cuentas dedicadas `bench_*`.

| ID | Nombre / Alias         | Rol asignado        | Email                    | Estado |
|----|------------------------|---------------------|--------------------------|--------|
| 1  | Administrador          | Administrador       | admin@mecanicgo.com      | Activo |
| 9  | Soportes Tigre         | Soporte técnico     | trigresoporte@gamial.com | Activo |
| 12 | Cesar Ticona Aguilar   | Mecánico automotriz | cesarticona@gmail.com    | Activo |
| 13 | Daniel Calisaya Mamani | Mecánico automotriz | Danielcalisaya@gmail.com | Activo |
| 14 | Leydy Ticona Canaza    | Cliente             | ydyelcanaza@gmail.com    | Activo |
| 15 | Yimmy Mamani Pari      | Mecánico automotriz | yimmymamani@gmail.com    | Activo |
| 16 | Marco Mamani Rodriguez | Mecánico automotriz | marcomamani@gmail.com    | Activo |
| 17 | Antonio Ccama Maquera  | Mecánico automotriz | antonioccama@gmail.com   | Activo |
| 18 | Mario Casas Alvarez    | Mecánico automotriz | mariocasas@gmail.com     | Activo |
| 19 | Jose Canaza Mamani     | Mecánico automotriz | josecanaza@gmail.com     | Activo |
| 20 | Julio Cauna Oliva      | Mecánico automotriz | juliocauna@gmail.com     | Activo |
| 21 | Ricardo Calcina Apaza  | Mecánico automotriz | ricardocalcina@gmail.com | Activo |
| 22 | Diego Salas Arpasi     | Mecánico automotriz | diegosalas@gmail.com     | Activo |

Tabla 5: Usuarios reales registrados en la base de datos.

**Total de usuarios reales registrados: 13** (1 admin, 1 soporte, 10 mecánicos, 1 cliente).

**Cuentas técnicas dedicadas al benchmark.** Para los escenarios POST (autenticación masiva, creación de citas y flujo completo) se usaron dos cuentas dedicadas que no interfieren con los datos de los usuarios reales:

| ID | Nombre / Alias     | Rol asignado        | Email                        | Estado |
|----|--------------------|---------------------|------------------------------|--------|
| 23 | Benchmark Cliente  | Cliente             | bench_cliente@mecanicgo.com  | Activo |
| 24 | Benchmark Mecanico | Mecánico automotriz | bench_mecanico@mecanicgo.com | Activo |

Tabla 6: Cuentas técnicas dedicadas a las pruebas.

## 11 Evidencias de los escenarios ejecutados

*Entregable 4 de la guía. La guía exige reportes o capturas de al menos 5 de los 10 escenarios; en esta entrega se incluyen los 10.*

Cada escenario cuenta con su reporte en formato Apache Benchmark (texto plano, idéntico al que emite el binario `ab.exe`) en la carpeta `benchmark/evidencias/`. Además, cada escenario tiene un archivo JSON con las métricas crudas y un CSV con todas las latencias individuales en `benchmark/resultados/`.

| Escenario                   | Reporte (formato ab)                          | JSON crudo                                     | CSV de latencias                                  |
|-----------------------------|-----------------------------------------------|------------------------------------------------|---------------------------------------------------|
| 1. Carga mínima             | evidencias/esc01_carga_minima.txt             | resultados/esc01_carga_minima.json             | resultados/esc01_carga_minima_lat.csv             |
| 2. Carga normal             | evidencias/esc02_carga_normal.txt             | resultados/esc02_carga_normal.json             | resultados/esc02_carga_normal_lat.csv             |
| 3. Carga moderada           | evidencias/esc03_carga_moderada.txt           | resultados/esc03_carga_moderada.json           | resultados/esc03_carga_moderada_lat.csv           |
| 4. Carga alta pico          | evidencias/esc04_carga_alta_pico.txt          | resultados/esc04_carga_alta_pico.json          | resultados/esc04_carga_alta_pico_lat.csv          |
| 5. Autenticación masiva     | evidencias/esc05_autenticacion_masiva.txt     | resultados/esc05_autenticacion_masiva.json     | resultados/esc05_autenticacion_masiva_lat.csv     |
| 6. Consulta a base de datos | evidencias/esc06_consulta_a_base_de_datos.txt | resultados/esc06_consulta_a_base_de_datos.json | resultados/esc06_consulta_a_base_de_datos_lat.csv |
| 7. Creación de recursos     | evidencias/esc07_creacion_de_recursos.txt     | resultados/esc07_creacion_de_recursos.json     | resultados/esc07_creacion_de_recursos_lat.csv     |
| 8. Carga sostenida          | evidencias/esc08_carga_sostenida.txt          | resultados/esc08_carga_sostenida.json          | resultados/esc08_carga_sostenida_lat.csv          |
| 9. Endpoint REST JSON       | evidencias/esc09_endpoint_rest_json.txt       | resultados/esc09_endpoint_rest_json.json       | resultados/esc09_endpoint_rest_json_lat.csv       |
| 10. Flujo completo          | evidencias/esc10_flujo_completo.txt           | resultados/esc10_flujo_completo.json           | resultados/esc10_flujo_completo_lat.csv           |

Tabla 7: Inventario de archivos de evidencia por escenario.

## 12 Enlaces a los reportes en el repositorio (Git)

*Trazabilidad de la entrega.* Todos los reportes, datos crudos y gráficas que sustentan este informe están versionados en el repositorio del proyecto y fijados al *commit* de la entrega, de modo que los enlaces apuntan siempre a la misma versión de cada archivo. El enlace base a la carpeta completa es:

<https://github.com/Leydy-Ticona-Canaza/Taller-de-Desarrollo-de-Software>

Específicamente en el apartado de "benchmark"

| #  | Escenario                | Reporte (enlace)                                   |
|----|--------------------------|----------------------------------------------------|
| 01 | Carga mínima             | <a href="#">esc01_carga_minima.txt</a>             |
| 02 | Carga normal             | <a href="#">esc02_carga_normal.txt</a>             |
| 03 | Carga moderada           | <a href="#">esc03_carga_moderada.txt</a>           |
| 04 | Carga alta / pico        | <a href="#">esc04_carga_alta_pico.txt</a>          |
| 05 | Autenticación masiva     | <a href="#">esc05_autenticacion_masiva.txt</a>     |
| 06 | Consulta a base de datos | <a href="#">esc06_consulta_a_base_de_datos.txt</a> |
| 07 | Creación de recursos     | <a href="#">esc07_creacion_de_recursos.txt</a>     |
| 08 | Carga sostenida          | <a href="#">esc08_carga_sostenida.txt</a>          |
| 09 | Endpoint REST JSON       | <a href="#">esc09_endpoint_rest_json.txt</a>       |
| 10 | Flujo completo           | <a href="#">esc10_flujo_completo.txt</a>           |

Tabla 8: Enlace al reporte de evidencia (formato ab) por escenario.

## 12.1 Datos crudos por escenario (JSON y CSV)

Para cada escenario se incluye además el archivo JSON con las métricas crudas y el CSV con todas las latencias individuales.

| #  | Resultado (JSON)                                    | Latencias (CSV)                                        |
|----|-----------------------------------------------------|--------------------------------------------------------|
| 01 | <a href="#">esc01_carga_minima.json</a>             | <a href="#">esc01_carga_minima_lat.csv</a>             |
| 02 | <a href="#">esc02_carga_normal.json</a>             | <a href="#">esc02_carga_normal_lat.csv</a>             |
| 03 | <a href="#">esc03_carga_moderada.json</a>           | <a href="#">esc03_carga_moderada_lat.csv</a>           |
| 04 | <a href="#">esc04_carga_alta_pico.json</a>          | <a href="#">esc04_carga_alta_pico_lat.csv</a>          |
| 05 | <a href="#">esc05_autenticacion_masiva.json</a>     | <a href="#">esc05_autenticacion_masiva_lat.csv</a>     |
| 06 | <a href="#">esc06_consulta_a_base_de_datos.json</a> | <a href="#">esc06_consulta_a_base_de_datos_lat.csv</a> |
| 07 | <a href="#">esc07_creacion_de_recursos.json</a>     | <a href="#">esc07_creacion_de_recursos_lat.csv</a>     |
| 08 | <a href="#">esc08_carga_sostenida.json</a>          | <a href="#">esc08_carga_sostenida_lat.csv</a>          |
| 09 | <a href="#">esc09_endpoint_rest_json.json</a>       | <a href="#">esc09_endpoint_rest_json_lat.csv</a>       |
| 10 | <a href="#">esc10_flujo_completo.json</a>           | <a href="#">esc10_flujo_completo_lat.csv</a>           |

Tabla 9: Datos crudos por escenario (resultados y latencias).

## 12.2 Gráficas y material global del benchmark

Por último, se enlazan las gráficas generadas y el material global del benchmark.

| Recurso                 | Enlace                             |
|-------------------------|------------------------------------|
| Gráfica RPS             | <a href="#">01_rps.png</a>         |
| Gráfica latencias       | <a href="#">02_latencias.png</a>   |
| Gráfica errores         | <a href="#">03_errores.png</a>     |
| Gráfica throughput      | <a href="#">04_throughput.png</a>  |
| Gráfica carga sostenida | <a href="#">05_sostenida.png</a>   |
| Gráfica comparativa     | <a href="#">06_comparativa.png</a> |
| Resumen global (JSON)   | <a href="#">resumen.json</a>       |
| Scripts del benchmark   | <a href="#">benchmark/scripts</a>  |

Tabla 10: Gráficas y material global del benchmark.